Text Preprocessing for NLP: A Comprehensive Guide

This document explains various text preprocessing techniques commonly used in Natural Language Processing (NLP). The document demonstrates these techniques using the IMDB movie reviews dataset, which contains 50,000 movie reviews labeled as either positive or negative sentiment.

Table of Contents

1. Introduction
2. Setting Up
3. Text Preprocessing Techniques
   - Lower Casing
   - Removing HTML Tags
   - Removing URLs
   - Removing Punctuation
   - Removing Stop Words
   - Handling Emojis
4. Text Tokenization
   - Using Split Function
   - Using Regular Expressions
   - Using NLTK
5. Stemming
6. Lemmatization
7. Feature Extraction Techniques
   - Count Vectorizer
   - TF-IDF Vectorizer
8. Conclusion

Introduction

Text preprocessing is a crucial step in any NLP project. Raw text data often contains noise, inconsistencies, and irrelevant information that can negatively impact model performance. Preprocessing helps clean and standardize the text data, making it suitable for analysis and machine learning algorithms.

The notebook demonstrates various preprocessing techniques using the IMDB dataset, which contains movie reviews with sentiment labels (positive or negative).

Setting Up

The notebook begins by importing pandas and loading the IMDB dataset:

```
import pandas as pd
df = pd.read_csv("/kaggle/input/imdb-dataset-of-50k-movie-reviews/IMDB Dataset.csv")
```

The dataset contains two columns: - `review`: The text of the movie review - `sentiment`: The sentiment label (positive or negative)

Text Preprocessing Techniques

Lower Casing

Converting all text to lowercase is one of the simplest preprocessing steps. This helps in standardizing the text and ensures that words like “Good”, “good”, and “GOOD” are treated as the same word.

```
df['review'] = df['review'].str.lower()
```

Removing HTML Tags

Many web-scraped texts contain HTML tags that don’t contribute to the meaning of the text. The notebook uses regular expressions to remove these tags:

```
import re  # Regular Expressions
def remove_html_tags(text):
    pattern = re.compile('<.*?>')
    return pattern.sub(r'', text)

df['review'] = df['review'].apply(remove_html_tags)
```

This function uses a regular expression pattern `<.*?>` to match HTML tags and replaces them with an empty string.

Removing URLs

URLs in text often don’t contribute to the meaning and can be removed:

```
def remove_url(text):
    pattern = re.compile(r'https?://\S+|www\.\S+')
    return pattern.sub(r'', text)

df['review'] = df['review'].apply(remove_url)
```

This function uses a regular expression to identify and remove URLs from the text.

Removing Punctuation

Punctuation marks are usually not needed for text analysis and can be removed:

```
import string
exclude = string.punctuation  # This contains: !"#$%&'()*+,-
./:;<=>?@[\]^_`{|}~

def remove_punc1(text):
    return text.translate(str.maketrans('','',exclude))

df['review'] = df['review'].apply(remove_punc1)
```

The `string.punctuation` contains all standard punctuation characters, and the `translate` method with `str.maketrans` is used to remove them from the text.

Removing Stop Words

Stop words are common words like "the", "is", "in" that typically don't carry much meaning. Removing them can help focus on the more informative words:

```
from nltk.corpus import stopwords

def remove_stopwords(text):
    new_text = []

    for word in text.split():
        if word in stopwords.words('english'):
            new_text.append('')
        else:
            new_text.append(word)
    x = new_text[:]
    new_text.clear()
    return " ".join(x)

df['review'] = df['review'].apply(remove_stopwords)
```

This function uses NLTK's stopwords list to identify and remove common English stop words.

Handling Emojis

The notebook shows two approaches to handling emojis:

1. **Removing emojis**:

```
import re
def remove_emoji(text):
    emoji_pattern = re.compile("["
                               "u\U0001F600-\U0001F64F"  # Emotions
                               "u\U0001F300-\U0001F5FF"  # Symbols &
pictographs
                               "u\U0001F680-\U0001F6FF"  # Transport & map
symbols
                               "u\U0001F1E0-\U0001F1FF"  # Flags(iOS)
```

```
                                "u\U00002702-\U000027B0"
                                "u\U000024C2-\U0001F251"
                                "]+", flags=re.UNICODE)
    return emoji_pattern.sub(r'', text)
```

2. **Converting emojis to text representation**:

```
import emoji
print(emoji.demojize('Python is 🔥'))  # Output: Python is :fire:
```

Text Tokenization

Tokenization is the process of breaking text into smaller units (tokens), typically words or sentences. The notebook demonstrates three different tokenization approaches:

Using Split Function

The simplest method is using Python's built-in `split()` function:

```
# Word Tokenization
a = 'I am going to delhi'
a.split()  # Output: ['I', 'am', 'going', 'to', 'delhi']

# Sentence Tokenization
b = 'I am going to delhi. I will stay there for 3 days. Let\'s hope the trip
to be great'
b.split('.')  # Output: ['I am going to delhi', ' I will stay there for 3
days', ' Let\'s hope the trip to be great']
```

However, this approach has limitations, especially for sentence tokenization where punctuation other than periods might denote sentence boundaries.

Using Regular Expressions

Regular expressions offer more flexibility for tokenization:

```
import re

# Word tokenization
e = 'I am going to delhi'
tokens = re.findall("[\w']+", e)  # Output: ['I', 'am', 'going', 'to',
'delhi']

# Sentence tokenization
text = """Lorem Ipsum is simply dummy text of the printing and typesetting
industry?
Lorem Ipsum has been the industry's standard dummy text ever since the 1500s,
when an unknown printer took a galley of type and scrambled it to make a type
specimen book."""
sentences = re.compile('[.!?] ').split(text)
```

Using NLTK

NLTK provides specialized tokenizers that handle various edge cases:

```
from nltk.tokenize import word_tokenize, sent_tokenize

# Word tokenization
sent1 = 'I am going to visit delhi!'
word_tokenize(sent1)  # Output: ['I', 'am', 'going', 'to', 'visit', 'delhi',
'!']

# Sentence tokenization
text = """Lorem Ipsum is simply dummy text of the printing and typesetting
industry?
Lorem Ipsum has been the industry's standard dummy text ever since the 1500s,
when an unknown printer took a galley of type and scrambled it to make a type
specimen book."""
sent_tokenize(text)
```

NLTK's tokenizers handle more complex cases like: - Abbreviations with periods (e.g., "Ph.D") - Contractions (e.g., "We're") - Special characters (e.g., "$", "@")

Stemming

Stemming is the process of reducing words to their word stem or root form. For example, the stem of "running", "runs", and "ran" is "run". The notebook uses Porter Stemmer from NLTK:

```
from nltk.stem.porter import PorterStemmer

ps = PorterStemmer()
def stem_words(text):
    return " ".join([ps.stem(word) for word in text.split()])

sample = "walk walks walking walked"
stem_words(sample)  # Output: 'walk walk walk walk'
```

While stemming is computationally efficient, it can sometimes produce stems that are not actual words, as shown in the example:

```
text = "probably my all time favourite movie a story of selflessness
sacrifice and dedication to a noble cause but its not preachy or boring it
just never gets old despite my having seen some 15 or more times in the last
25 years."
stem_words(text)
# Output includes words like 'probabl', 'favourit', 'movi', 'sacrific', etc.
```

Lemmatization

Lemmatization is similar to stemming but produces actual dictionary words. It's more sophisticated and considers the context and part of speech of words:

```
from nltk.stem import WordNetLemmatizer

wordnet_lemmatizer = WordNetLemmatizer()

sentence = "He was running and eating at same time. He has bad habit of
swimming after playing long hours in the Sun."
punctuations = "?:!.,;"
sentence_words = nltk.word_tokenize(sentence)
for word in sentence_words:
    if word in punctuations:
        sentence_words.remove(word)

print("{0:20}{1:20}".format("Word","Lemma"))
for word in sentence_words:

print("{0:20}{1:20}".format(word,wordnet_lemmatizer.lemmatize(word,pos='v')))
```

Lemmatization is slower than stemming but typically produces better results, especially for applications where readability is important.

Feature Extraction Techniques

After preprocessing the text, we need to convert it into a numerical format that machine learning algorithms can understand. Two common techniques for this are Count Vectorizer and TF-IDF Vectorizer.

Count Vectorizer

Count Vectorizer converts text documents into a matrix of token counts. Each row represents a document, and each column represents a unique word in the corpus. The values in the matrix represent the frequency of each word in each document.

```
from sklearn.feature_extraction.text import CountVectorizer

# Example text corpus
corpus = [
    'This is the first document.',
    'This document is the second document.',
    'And this is the third one.',
    'Is this the first document?',
]

# Create a CountVectorizer
vectorizer = CountVectorizer()
```

```
# Transform the corpus
X = vectorizer.fit_transform(corpus)

# Get feature names (unique words)
print(vectorizer.get_feature_names_out())
# Output: ['and', 'document', 'first', 'is', 'one', 'second', 'the', 'third',
'this']

# Print the document-term matrix
print(X.toarray())
'''
Output:
[[0 1 1 1 0 0 1 0 1]
 [0 2 0 1 0 1 1 0 1]
 [1 0 0 1 1 0 1 1 1]
 [0 1 1 1 0 0 1 0 1]]
'''
```

The matrix shows, for example, that the word “document” (index 1) appears once in document 0, twice in document 1, zero times in document 2, and once in document 3.

TF-IDF Vectorizer

Term Frequency-Inverse Document Frequency (TF-IDF) is a statistical measure that evaluates how relevant a word is to a document in a collection of documents. It increases proportionally to the number of times a word appears in the document but is offset by the frequency of the word in the corpus.

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Create a TF-IDF Vectorizer
tfidf_vectorizer = TfidfVectorizer()

# Transform the corpus
X_tfidf = tfidf_vectorizer.fit_transform(corpus)

# Get feature names
print(tfidf_vectorizer.get_feature_names_out())
# Output: ['and', 'document', 'first', 'is', 'one', 'second', 'the', 'third',
'this']

# Print the TF-IDF matrix
print(X_tfidf.toarray())
'''
Output (values are approximate):
[[0.   0.38 0.46 0.38 0.   0.   0.38 0.   0.38]
 [0.   0.64 0.   0.30 0.   0.46 0.30 0.   0.30]
 [0.43 0.   0.   0.33 0.43 0.   0.33 0.43 0.33]
 [0.   0.38 0.46 0.38 0.   0.   0.38 0.   0.38]]
'''
```

In the TF-IDF matrix, words that are common across all documents (like “is” and “the”) receive lower weights, while words that are unique to specific documents (like “second” or “third”) receive higher weights. This helps to highlight the more distinctive or important words in each document.

How TF-IDF Works

TF-IDF consists of two components:

1. **Term Frequency (TF)**: The frequency of a word in a document.
   - TF(t, d) = (Number of times term t appears in document d) / (Total number of terms in document d)
2. **Inverse Document Frequency (IDF)**: The importance of a word across all documents.
   - IDF(t) = log_e(Total number of documents / Number of documents containing term t)
3. **TF-IDF**: The product of TF and IDF.
   - TF-IDF(t, d) = TF(t, d) * IDF(t)

TF-IDF gives higher weights to words that are frequent in a specific document but rare across the corpus, making it useful for: - Identifying distinctive terms in documents - Reducing the impact of common words - Information retrieval and text mining - Document similarity calculations

Conclusion

Text preprocessing is a critical step in any NLP project. The techniques described in this document help clean and standardize text data, making it suitable for analysis and machine learning algorithms. After preprocessing, feature extraction methods like Count Vectorizer and TF-IDF convert the text into numerical format for model training.

Different NLP tasks may require different combinations of these techniques, and it’s important to experiment to find the optimal preprocessing pipeline for your specific application. For example, some tasks may benefit from keeping stop words, while others might require lemmatization instead of stemming.

Understanding these foundational techniques provides a solid basis for more advanced NLP applications such as sentiment analysis, text classification, topic modeling, and more.